



PRODUCT & TECHNICAL DOCUMENTATION · DEEP DIVE

Bolt Sentinel

An AI-native Regulatory Operations Platform for ride-hailing — one integrated system covering Authority Request Response, Fleet Partner Onboarding, Ongoing Compliance Monitoring, and Regulation Intake over a shared compliance-intelligence engine.

4 operational surfaces

15 AI agents

6 markets · GDPR / NDPA / POPIA

Human-in-the-loop governance

Hash-chained audit

Contents

I. Product Overview

- What Bolt Sentinel is
- The problem it solves
- The four operational surfaces at a glance
- The value contrast: hours to seconds
- Governance non-negotiables

II. The Operational Surfaces (Feature Deep-Dive)

- Surface A — Authority Request Response
- Surface B1 — Fleet Partner Onboarding
- Surface B2 — Ongoing Compliance Monitoring
- Surface C — Regulation Intake
- Shared surfaces: Home, Assistant, Markets & Rules, Accuracy, Audit, Settings, Guide

III. The AI Engine

- AI-first principle & the centralized client
- The 15 agents
- Model routing & tiers
- Prompt registry & the 17 prompts
- Cost ledger

IV. Shared Core Architecture

- Engine module map
- The re-entrant pipeline pattern & runner
- Governance state machines
- Provenance system
- Hash-chained audit log
- Document storage driver

V. The Two Critical Subsystems

- §7.A Compliance-Constraint Framework
- §7.B Accuracy & Confidence Layer

VI. The Market Ruleset System

- Market = ruleset abstraction
- The MarketRuleset contract
- Validation DSL & executor capability surface
- The six markets in detail

VII. Data Model

VIII. API Reference

IX. Frontend, Design System & UX

- X. RBAC, Authentication & Security**
- XI. Synthetic Data, Seed & Scenarios**
- XII. Tech Stack, Deployment & Operations**
- XIII. Appendices — Enums, Glossary & File Map**

PART I

Product Overview

What Bolt Sentinel is, the manual-compliance problem it removes, the four surfaces it ships, and the governance principles that bind everything it does.

1.1 What Bolt Sentinel is

Bolt Sentinel is an AI-native **Regulatory Operations Platform** for a ride-hailing super-app operating across six markets — Estonia (Tallinn), Poland (Warsaw), Portugal (Lisbon/Porto), Romania (Bucharest), Nigeria (Lagos), and South Africa (Johannesburg/Cape Town). It is a single, integrated, production-grade platform that automates the highest-effort manual compliance work in ride-hailing central operations.

It presents to users as one platform with four operational surfaces sitting on a shared compliance-intelligence core. Each surface follows the same computational shape — *ingest unstructured input* → *interpret it against a market-specific regulatory ruleset* → *validate or retrieve the relevant data* → *produce a governed, fully-traceable output* — with explainability on every field, an immutable audit trail, and a human decision required before anything binds.

SURFACE	WHAT IT DOES	WHO USES IT
A — Authority Request Response (ARR)	Interpret a regulator's request, map it to that market's legal requirements, retrieve the data, and produce a compliant, traceable report for human review and submission.	Local operations / compliance staff
B1 — Fleet Partner Onboarding	Validate a partner's document pack against market rules, cross-check across documents, risk-score, and draft a go/no-go decision for a human.	Fleet operations / compliance staff
B2 — Ongoing Compliance Monitoring	Continuously watch the live partner portfolio for expiries and drift, re-validate on renewal, and proactively re-trigger human review.	Fleet operations / compliance monitors
C — Regulation Intake	Upload a regulation; AI reads it, classifies the jurisdiction, and proposes a complete executable market ruleset that a human reviews and activates — this is how rules stay current.	Regulatory authors / compliance leads

Everything runs on realistic **synthetic data** across the six markets. The product is branded "Bolt Sentinel" in Bolt's design language for a private demo only.

IP GUARDRAIL

The "Bolt Sentinel" name, the green palette, and the lightning-bolt mark are an *approximation* of Bolt's design language recreated for a private demo shown directly to Bolt. The product is not affiliated with or endorsed by Bolt; the mark is an original SVG recreation, not Bolt's proprietary logo. The deployment is `noindex`. For any public use the mark/name should be swapped for a neutral brand labelled "concept built for a Bolt application."

1.2 The problem it solves

A ride-hailing platform operating across many countries faces an enormous, non-uniform regulatory surface. Every market has its own transport-licensing regime, document requirements for fleet partners and drivers, data-residency/privacy/retention/lawful-basis constraints (GDPR variants across the EU; NDPA in Nigeria; POPIA in South Africa), and authority reporting obligations. Two workflows consume the most manual effort.

Problem A — Authority Request Response

Local operations teams receive regular, ad-hoc requests from regulators ("provide all drivers who operated in central Lisbon during Q1 2026, with license validity and total trips"). Today an ops person must interpret what is actually being asked, know the market-specific legal requirements and acceptable format, figure out which internal systems hold the data, pull and cross-reference it, assemble a compliant report, and get it reviewed. This takes hours to days per request, recurs across many markets, and carries legal and financial risk if done wrong or late. The knowledge of "how to answer this authority in this market" lives in local staff heads and does not scale.

Problem B — Fleet Partner Compliance

A fleet partner is a company running multiple vehicles and drivers on the platform. Onboarding (B1) means collecting and validating a market-specific document set — operator licenses, commercial insurance, vehicle registrations and inspection certificates, driver certifications, tax/business registration — for authenticity, completeness and internal consistency, then making a go/no-go call. Monitoring (B2) is the fact that compliance is not static: documents expire on rolling schedules, fleets change, regulations update. Lapses today go unnoticed until they become incidents.

WHY ONE PLATFORM

All four surfaces are computationally the same shape, and the market-rule layer, document/text understanding, governance/audit/human-in-the-loop layer, and explainability + accuracy layer are common infrastructure. Building them once and serving multiple compliance workflows is simply how a serious platform is built. To the user it is one integrated regulatory-operations platform.

1.3 The four operational surfaces at a glance

Surfaces A, B1, and C are **re-entrant pipelines** driven by one runner; B2 is a continuous monitoring loop over the live portfolio that reuses B1's validation machinery. Each pipeline runs live Claude calls one stage at a time, persisting output, confidence and timing so the UI streams progress.

SURFACE	PIPELINE STAGES (IN ORDER)	STATE MACHINE END STATES
A — ARR	intake → rule-mapping → retrieval-plan → retriev-al-exec → generation → self-validation	APPROVED_FOR_SUBMISSION / REJECTED / NEEDS_CLARIFICATION
B1 — On-boarding	extraction → validation & cross-checks → risk → decision draft	APPROVED / CONDITIONS_APPLIED / REJECTED
B2 — Monitor-ing	(loop) signals → ruleset re-flag → drift → expiry forecast → status	REMEDIATED / CONDITIONS_APPLIED / SUSPEN-DED_RECOMMENDED
C — Regula-tion Intake	read → classify → synthesize → self-validate → diff → finalize	ACTIVATED / REJECTED / NEEDS_EDIT

1.4 The value contrast: hours to seconds

The persuasion device is threaded throughout the product: "this took a person half a day; watch it happen in seconds." Manual baselines are configurable per workflow (ARR default 240 minutes, onboarding 360, regulation intake 480) and clearly labelled estimates. Sentinel surfaces the contrast in several places:

- **Per-item completion** — a "Manual baseline ~Xh / Sentinel Ys" banner on each completed request or onboarding, using the real elapsed AI processing time (`aiElapsedMs`).
- **Per-stage timing** — the pipeline stepper shows how long each AI stage took.
- **Portfolio level** — the Home dashboard shows cumulative hours saved this period plus the total AI cost (the capital-efficiency story: hours saved against dollars spent).
- **Audit/report footer** — elapsed time, model used, and cost per artifact for traceability.

1.5 Governance non-negotiables

Six principles are enforced architecturally, not bolted on. They appear verbatim in the project's `CLAUDE.md` as the rules the system must never violate.

PRINCIPLE	HOW IT IS ENFORCED
Human-in-the-loop before anything binds	The system never auto-submits to an authority, never auto-approves/rejects a partner, never auto-suspends. AI proposes; a human decides. Enforced by the state machines in <code>engine/governance/</code> .
Provenance on every generated figure	Every number traces to its exact source rows plus the prompt version that produced it, and is clickable in the UI.
The LLM never touches the database	Planning agents emit a typed query plan; a deterministic executor is the only thing that runs Prisma. Figures are computed by code, so provenance is real.
Audit everything, immutably	Every meaningful action is appended to a hash-chained audit log with actor, before/after, and — for AI — model, prompt version, cost and confidence.
AI everywhere it adds value	No deterministic stubs where reasoning belongs; multi-tier routing keeps it cost-disciplined.
Compliance + accuracy are architectural	The §7.A compliance-constraint framework and §7.B accuracy/confidence layer are enforced and visible.
Market = ruleset	Every rule lives in the DB as a Zod-validated <code>MarketRuleset</code> ; adding/changing a market is a new ruleset version, never a change to engine code.

PART II

The Operational Surfaces

A feature-by-feature deep dive into each of the four surfaces and every shared screen — what they do, the pipeline behind them, every panel and interaction, and the sub-features.

2.1 Surface A — Authority Request Response (ARR)

A regulator sends a request; Sentinel interprets it, maps it to the market's legal requirements, retrieves the data deterministically, generates a compliant report with per-figure provenance, self-validates it, and hands a review-ready artifact to a human. The system never submits to the authority — a human approves, and an export is produced for them to submit.

The end-to-end flow

A request is created with a reference like `ARR-EE-0007`, a title, the authority, optional cited legal basis, the raw text (or a source PDF), a market, a deadline, and a manual baseline. It starts in status `RECEIVED` and runs through a six-stage pipeline (surface `ARR`):

#	STAGE	AGENT / TIER	WHAT HAPPENS
1	Intake → Intent	IntakeAgent · balanced · <code>intake.request.v1</code>	Parses the free text/PDF into a structured <code>RequestIntent</code> : summary, authority, legal basis, purpose, subject types (driver/vehicle/fleet partner/trip), requested fields, filters (zone, period, status), deadline, ambiguities, and a confidence. Gate: if any ambiguity is found (only ever raised when the reporting <i>period</i> is undeterminable, e.g. "the period referenced in our earlier letter") the request transitions to <code>NEEDS_CLARIFICATION</code> and the pipeline blocks for a human.
2	Rule mapping → Compliance plan	RuleMappingAgent · reasoning · <code>rules.interpret.v1</code>	Reasons over the market ruleset (passed as cacheable context) to classify each requested field as ANSWERABLE , CAUTIONED (needs minimization/aggregation), or OUT_OF_SCOPE (over-reach / restricted without basis). Produces the mandated report format id, a compliance checklist, and structure notes.
3	Retrieval planning	RetrievalPlannerAgent · balanced · <code>retrieval.plan.v1</code>	Turns the plan into a typed query plan — one query per answerable/cautioned field — using only the ruleset's field keys. It does <i>not</i> touch the database.
4	Retrieval + compliance	Deterministic (no LLM) + ComplianceConstraintAgent	The deterministic executor runs Prisma queries for every non-out-of-scope field, tagging each figure with its source row ids (real provenance). Then the §7.A constraint framework enforces residency/PII/lawful-basis rules — masking, aggregating, or withholding fields — recording an explainable decision per field.
5	Report generation	GenerationAgent · reasoning · <code>generation.report.v1</code>	Generates the compliant report in the mandated language and structure, using only the computed dataset and copying values exactly. Persists a normalized <code>ReportField</code> row per figure (with provenance + the originating AI call id) for the click-through provenance UI.
6	Self-validation + confidence	Deterministic verify + ValidationAgent · reasoning · <code>validation.selfcheck.v1</code>	First, a deterministic figure-vs-source check recomputes every figure against its cited rows. Then an adversarial validation pass confirms each checklist item is satisfied and each figure is plausible, flagging blocking discrepancies. Confidence per field is attached; the run completes to <code>PENDING_REVIEW</code> .

On human approval the status becomes `APPROVED` (approved for submission) and an export is produced. Other human actions: reject, request changes (moves to `NEEDS_CLARIFICATION` with a note), regenerate (re-runs the pipeline), and clarify (appends a clarification note to the raw text and re-runs).

The request inbox

Lists non-deleted requests scoped to the active market filter, ordered by received date with open statuses floated to the top; the header shows an open count. Columns: reference (mono link), request (title + authority), market, status badge, and a live SLA countdown chip that turns amber within 24 hours of the deadline and red when overdue.

The request workspace

The detail screen is the core ARR experience. A header shows the reference, title, country · authority · legal basis, the SLA chip, and the status. Conditional banners appear for `NEEDS_CLARIFICATION` (clarification panel), terminal decisions (a decision-outcome banner with reviewer name and timestamp), and a time-saved banner once the run has elapsed.

- **Left rail** — the live *pipeline stepper* (starts the pipeline, polls status every 700ms and drives one stage advance at a time, expandable per-stage output and confidence); the original *authority request* text; the *AI interpretation* panel (summary, purpose, subjects, zone, period, ambiguity warnings); and the *compliance checklist* panel where each item carries a satisfied/**X** icon cross-referenced to the self-validation result, plus a "field disposition" view mapping ANSWERABLE → PASS, OUT_OF_SCOPE → FAIL, CAUTIONED → WARN.
- **Right (main)** — the *generated report* rendered as a styled article. Each `{{fieldKey}}` token in a section body is rendered as a clickable *provenance figure*. Review actions sit at the top. Below: a "compliance constraints applied" panel (each §7.A decision as allow → PASS / block → FAIL / mask·aggregate → WARN with rule + rationale), a "supporting detail" panel for list-typed fields, and a "self-validation" panel (figures-verified count, an overall confidence meter, and a blocking warning if present).

Provenance figures (click-through to source)

Every figure is an inline button (dashed underline; brand-green with a ✓ if verified, amber with a ⚠ if not). Opening it fetches `/api/report-fields/[id]/provenance`, which resolves the figure's provenance refs into the real source rows (Driver/Vehicle/Trip/FleetPartner, capped at 50 ids each) and returns the originating AI call's model, prompt version, cost and confidence. The dialog shows the classification, total source-row count, the AI call metadata, and a mini-table per source entity. This realizes the "every figure traces to its source rows + the prompt that produced it" guarantee.

Report export

`GET /api/authority-requests/[id]/export?format=pdf|json` produces the submission artifact. The PDF (built with pdfkit, `lib/report-pdf.ts`) carries a Bolt-green header bar with the bolt mark and wordmark, regulator/market/reference, the title, preamble, sections (with `{{key}}` figure substitution), a "Reported figures" table, the closing, and a footer noting generation time/model plus "Synthetic demonstration data — not an official filing."

ARR SCENARIOS SEEDED

Six requests demonstrate distinct behaviours: a **clean Tallinn** Q1 driver/trip request (happy path), an **ambiguous Warsaw** request referencing an unspecified earlier letter (surfaces a clarification), a **Lagos NDPA** driver-PII roster (the constraint engine aggregates/masks), a **Johannesburg overreach** request for per-driver criminal records (flagged out-of-scope), and clean **Lisbon** and **Bucharest** requests showing market-specific report formats.

2.2 Surface B1 — Fleet Partner Onboarding

A partner submits a document pack; Sentinel reads every document natively, validates each against the market's rules, cross-checks across documents, risk-scores the partner, and drafts an approve / approve-with-conditions / reject decision for a human. The system never auto-decides.

The four-stage pipeline (surface B1)

#	STAGE	ENGINE	WHAT HAPPENS
1	Document extraction	ExtractionAgent · reasoning · extracted-fields.v1	For each document, the PDF/image bytes are fed to Claude natively (no separate OCR). Expected fields come from the ruleset's document spec. Per-field calibrated confidence is captured; illegible/absent fields are marked present=false. A document whose overall confidence < 0.70 (or any present field < 0.60) is FLAGGED rather than passed silently; otherwise EXTRACTED . Documents are processed in parallel.
2	Validation & cross-checks	Deterministic DSL (no LLM)	Each document's validation rules are evaluated with the rule DSL (13 operators — regex, date-not-expired, cross-field equals, numeric comparisons, etc.) producing PASS/WARN/FAIL with severity, message, and evidence. Cross-document checks compare fields across documents (e.g. operator-license holder vs business-registration name) producing MATCH/MISMATCH/INDETERMINATE. Completeness is computed as required non-optional docs present for the partner type, yielding a missing-docs list and a completeness %.
3	Risk scoring	RiskAgent · balanced · risk.score.v1	Aggregates signals (doc/vehicle/driver counts, FAIL/WARN counts, cross-check mismatches, low-confidence docs, items expiring within 30 days, expired items) and the market's risk model (weighted factors: completeness 0.30, expiry proximity 0.25, inconsistencies 0.30, fleet size 0.15). Emits a 0–100 score (higher = riskier), a band (LOW/MEDIUM/HIGH), and per-factor contributions with evidence. Supersedes the prior assessment.
4	Decision draft	GenerationAgent · reasoning · generation.onboarding.v1	Drafts a go/no-go: APPROVE / APPROVE_WITH_CONDITIONS / REJECT with conditions and rationale. A low-confidence extraction becomes a required human-verification condition. The pipeline completes to PENDING_REVIEW ; a human binds the decision.

The onboarding queue

Lists partners scoped to the market with counts of documents/vehicles/drivers; queue statuses sort to the top; the header reads "x awaiting · y total." Columns: partner (legal name + reference), market, type, fleet summary (vehicles · drivers · docs), completeness bar + %, risk band badge, and status.

The onboarding workspace

- **Documents panel** — a document viewer with a left list (FLAGGED ⚠ / VALIDATED ✓ icons), an iframe PDF preview of the original, and an "extracted fields" panel showing each field's value with a confidence meter; missing or low-confidence (< 0.70) fields are highlighted, notes are shown, and a "gaps" footer lists what the model could not extract.

- **Pipeline stepper** — runs the onboarding pipeline and streams stage progress.
- **Decision draft panel** — the AI's recommended outcome (hint badge while still a draft), rationale, conditions list and confidence meter, with reject / approve / approve-with-conditions actions.
- **Risk assessment panel** — score, band badge, explanation, and per-factor contribution bars.
- **Required-documents checklist** — "x/y" with present ✓ / missing ✗ per required doc for the partner type.
- **Validation & cross-checks panel** — cross-check messages (including "valueA ≠ valueB" on mismatch), non-passing validations, or "All N rules passed."

B1 SCENARIOS SEEDED

A **clean** Portugal partner (fast approve), a **name-mismatch** partner whose operator license and business registration disagree (caught by a cross-check), a **missing-document** Poland partner (checklist gap → approve-with-conditions draft), and a **low-confidence-extraction** partner whose illegible TVDE license forces mandatory human review via confidence gating.

2.3 Surface B2 — Ongoing Compliance Monitoring

Once a partner is onboarded, compliance is continuously monitored over the live portfolio. B2 is a distinct lifecycle with its own state machine and screens — it is not a re-entrant pipeline but a direct-call engine (`surfaces/fleet-monitoring/monitor.ts`) that reuses B1's validation machinery. It never auto-suspends; it routes drift to a human.

The monitoring sweep

`runMonitoringForPartner(partnerId)` runs the lifecycle for one partner; `runSweep(marketId?)` sweeps every APPROVED / CONDITIONS_APPLIED partner. The sweep is triggerable manually (the dashboard "Sweep" button → `POST /api/monitoring/sweep`) and on a Vercel cron (`/api/cron/monitoring-sweep`). For each partner:

1. **Deterministic signals** — builds an upcoming-expiry list (vehicle inspection/insurance/registration and driver licence within a 30-day horizon, sorted by days-until); detects vehicles added in the last 45 days with no valid registration; detects expired inspections; and computes whether the partner's last-assessed ruleset version is behind the market's active version.
2. **Ruleset re-flag (highest priority)** — if the partner is on an older ruleset version, status → `PENDING_REVIEW` and a `RULESET_REFLAG` compliance event is recorded.
3. **Drift detection** — if signals exist, the DriftDetectionAgent (balanced, `risk.drift.v1`) compares previous vs current state for new vehicles without docs, lapsed documents, and fleet-composition changes; on detected drift the status becomes `DRIFT_DETECTED` and a `DRIFT_DETECTED` event is recorded.
4. **Expiry forecast** — if upcoming expiries exist, the ExpiryForecastAgent (fast, `risk.forecast.v1`) contextualizes them into proactive gap predictions with severity (≤ 7 days CRITICAL, ≤ 21 HIGH, ≤ 30 MEDIUM); a still-compliant partner becomes `EXPIRING_SOON` with an `EXPIRY_FORECAST` event.
5. **Status application** — a human-owned `PENDING_REVIEW` is preserved; an AI `MONITORING_SWEEP` audit row is written.

Re-validation on renewal

`revalidatePartner(partnerId)` (`POST /api/partners/[id]/monitoring/revalidate`) re-extracts any newly-uploaded (renewed) documents using the same ExtractionAgent, sets them VALIDATED, extends expiring vehicle insurance where an insurance doc was renewed, re-runs the monitoring lifecycle, and — if the partner was EXPIRING_SOON — returns it to COMPLIANT. It records a resolved `REVALIDATION` event and partner-history entry. No full re-onboarding is needed.

The monitoring dashboard

Scope is APPROVED / CONDITIONS_APPLIED partners, market-filtered. Four KPIs: portfolio compliant %, at-risk partner count, items expiring within 12 weeks, and open alerts. Panels: a *portfolio health* stacked bar by monitoring status; an *expiry pipeline* bar chart bucketing vehicle inspection/insurance expiries into six two-week windows over 12 weeks; an *at-risk partners* table (risk, status, reason); and a *proactive alerts* list of open compliance events with severity.

The partner monitoring detail

Shows a document & vehicle validity table (plate plus inspection/insurance/registration dates coloured by validity — missing/expired FAIL, within 30 days WARN, else PASS), a merged compliance-history-and-events timeline, and the current risk breakdown. Human actions (`POST /api/partners/[id]/monitoring/decision`): re-validate, mark re-

mediated, apply conditions, recommend suspension, or clear back to compliant — each creating a partner-history entry and a resolved `HUMAN_ACTION` event.

B2 SCENARIOS SEEDED

An Estonia partner with two vehicle inspections lapsing in 21 days (expiry forecast + proactive alert + risk bump); an Estonia partner who added two vehicles without valid registration (drift detection flags it); a Poland partner whose insurance expires in 10 days with a hand-built "renewed" insurance PDF ready for re-validation; and the Estonia ruleset bump from v1 to v2 (adding a driver background-check document) which re-flags previously-compliant partners for review.

2.4 Surface C — Regulation Intake

This is how rules stay current. Upload a regulation (PDF or pasted text); AI reads it into quotable sections, classifies the jurisdiction, and synthesizes a complete, executable market ruleset constrained to what the engine can actually run. The draft is quarantined until a human activates it — at which point it becomes a live market or a new version, and affected partners are re-flagged.

The six-stage pipeline (surface REGINTAKE)

#	STAGE	AGENT / TIER	WHAT HAPPENS
1	Read regulation	RegulationReader-Agent · reasoning · <code>regulation.read.v1</code>	Reads the uploaded regulation natively and decomposes it into stable, quotable sections (the provenance source), detecting language (BCP-47), country, regulator and topics.
2	Classify jurisdiction	RegulationClassifier-Agent · balanced · <code>regulation.classify.v1</code>	Determines jurisdiction, region (EU/NG/ZA/US/OTHER), privacy regime (GDPR/NDPA/POPIA), currency/locale/timezone, and a suggested market code; decides whether it matches an existing market (a version update) or is a new market.
3	Synthesize ruleset	RulesetSynthesis-Agent · reasoning · <code>ruleset.synthesize.v1</code>	"The most consequential agent." Emits a full <code>MarketRuleset</code> using only the executor's closed vocabularies (entities, aggregations, operators). Anything it cannot express is surfaced as <code>unmapped-Fields</code> / <code>unsupportedClauses</code> — never invented. Each element carries provenance with a verbatim source quote and section id. For updates it keeps stable ids and emits change notes.
4	Validate + dry-run	Deterministic + ValidationAgent · reasoning · <code>ruleset.validate.v1</code>	Runs <code>MarketRulesetSchema.safeParse</code> (schema gate) plus <code>validateRulesetSources</code> (an "executor dry-run" confirming every answerable field's source is actually executable), then an adversarial review of faithfulness/completeness/coherence/privacy-posture. Blocking if the schema fails, sources are not executable, or the review flags critical/high issues.
5	Diff vs current	Deterministic (no LLM)	For an update, produces a structural diff vs the current active version (documents/fields/cross-checks added·removed·changed, plus policy changes); for a new market, marks it new.
6	Ready for review	—	Transitions the import to <code>PENDING_REVIEW</code> with the proposed market/version.

The intake screens

The list shows up to 100 imports with reference, source (filename or "pasted text"), proposed market (code + version + "new" chip), status chip, confidence, and date; an upload form (drag/drop PDF/PNG/JPEG or paste text) sits above. The detail screen shows the classification chips (country, region, privacy regime, target code+version, new-market vs version-update) and, once a draft exists: proposed required documents, authority-answerable fields, the proposed compliance policy, a "flagged for human wiring" panel (the honest unmapped fields and unsupported clauses), and a provenance panel (up to 40 entries with source quotes and section ids). The right rail holds the decision actions, a validation panel (schema valid / sources executable / blocking chips and issues), a diff panel for updates, and the classification rationale.

Activation — the only path into the rules table

`POST /api/regulation-intake/[id]/activate` is the sole way a synthesized ruleset becomes live. It re-parses the draft against `MarketRulesetSchema` (a hard gate), then either creates a new Market + RegulatoryRuleset (new market) or supersedes the prior active version and bumps `activeRulesetVersion` (update). On an update it **re-flags** every partner whose last-assessed version is behind the new one (creating RULESET_REFLAG events and moving them COMPLIANT → PENDING_REVIEW), invalidates the ruleset cache, and records a `RULE-SET_ACTIVATED` audit row. Reviewers can also save edits to the quarantined draft (`PATCH .../draft` , re-parsed against the schema) or reject it.

2.5 Shared surfaces

Home dashboard

A cross-surface, market-filterable operational view. Five KPI stats: open authority requests, onboarding queue size, portfolio compliant % (x of y), items expiring within 30 days, and cumulative hours saved (with total AI cost and call count as a sublabel). Two panels — open authority requests (with SLA chips) and "needs attention" at-risk partners — plus a "recent AI activity" feed showing each recent AI call's agent, stage, model, prompt version, confidence, latency and cost.

Conversational assistant (§J)

A global, grounded LLM assistant spanning all surfaces. It is opened by the "Ask" button, the §J shortcut, or the §K command palette. It builds a grounded snapshot scoped to the market filter (recent authority requests, at-risk partners, expiring vehicles, open compliance events, and the portfolio breakdown — each item carrying an href), and answers *only* from that context with citations; ungrounded questions get a visible "no grounded source" caveat rather than a hallucination. It streams token-by-token (`/api/assistant/stream`) and can return an inline chart (bar/line/donut) and follow-up suggestions. Citations become clickable record links with a live popover preview (`/api/assistant/preview`). Conversations are saved in localStorage; a dedicated Assistant page lists saved conversations alongside the chat thread.

Markets & Rules

A read-only explorer over the active rulesets. The list resolves each market's active ruleset and shows regulator, regime, region, cross-border posture, active version, content hash, currency, and doc/field/check/zone counts, filterable by regime, region, and residency. The detail screen renders the full ruleset: required documents (accordion with expected fields and validation rules), authority-answerable fields with classifications and caution notes, cross-document checks, the mandated report format, the risk model, all ruleset versions with changelogs and hashes, and the full §7.A compliance policy (data residency, retention & purpose, lawful bases, PII classification).

Accuracy view (§7.B)

Computes measured accuracy over the synthetic corpus from ground truth. Five KPIs: extraction F1, precision, recall, calibration error (ECE), and defect catch-rate. Panels: a calibration reliability view (predicted vs actual accuracy per confidence bucket with the ECE summary), a per-field confidence distribution chart keyed to the pass threshold, the confidence-gating policy (auto-pass ≥85% with higher bars for financial 90% / sensitive 92%, soft-flag 60–85%, hard block <60% or unverified/mismatch), and accuracy by market. When no data is present it still shows the methodology and gating policy.

Audit & Cost

Shows the latest audit entries with actor (AI/HUMAN/SYSTEM icon + name), action, entity, AI detail (model · prompt version · cost) and timestamp; a chain-integrity badge ("Verified" or "Broken @ N") from `verifyAuditChain()` ; the total AI cost and call count; tokens in/out; and cost grouped by model.

Settings

Nine tabs: General (workspace name, default market, manual baselines, theme), AI & Models (Anthropic key status + admin set/clear, per-tier model routing with rate-card pricing, prompt-caching toggle, monthly budget alert), Connectors & Data (datastore/storage status, intake channels, webhook, API ingest tokens), Approval Workflows

(the locked human-in-the-loop gates plus dual-approval policy toggles), Confidence & Accuracy (pass/review/financial/sensitive thresholds, block-on-unverified, block-on-mismatch), Compliance & Data (cross-border default, retention months, auto-purge), Team & Roles (team table + add teammate + the RBAC matrix), Notifications (six trigger toggles + channels), and Audit & Security (verify chain, session timeout, danger zone).

User Guide

A standalone, data-driven, searchable 14-section walkthrough (Getting Started, Roles & Access, each surface, Markets & Rules, Regulation Intake, Accuracy, Audit, Assistant, Settings, Reference & FAQ, Governance & Trust) with a scroll-spy sidebar, deep-linkable sections, callouts, step lists, tables, and status-lifecycle references.

PART III

The AI Engine

The AI-first principle, the single centralized Claude client, the fifteen named agents, multi-tier model routing, the versioned prompt registry, and the cost ledger.

3.1 AI-first principle & the centralized client

Every value-adding stage is a named **agent** — a typed LLM operation routed to a model tier and backed by a versioned prompt file. A single core invariant governs the whole system: **the LLM never queries the database**. Planning agents emit a typed query plan; a deterministic executor is the only Prisma caller and tags every row with its primary key. The model plans and narrates; figures are computed in code, so provenance is real.

Exactly one file imports the Anthropic SDK: `engine/llm/client.ts`. Every call goes through it so that cost, prompt version, latency and confidence are always recorded. Its `callLLM<T>()` function:

- Resolves the prompt by id/version from the registry, the model from the routed tier (overridable via Settings), and a default 4096 max tokens.
- Builds the system text (optionally appending a cacheable market-ruleset context block, marked ephemeral for prompt caching when enabled), and renders `{{var}}` templates.
- Attaches documents as native `document` (PDF) or `image` blocks — no separate OCR.
- Forces **Structured Outputs** by registering a tool whose input schema is the Zod schema converted to JSON Schema, with `tool_choice` pinned to it; the tool input is then `schema.parse()` d.
- Retries up to four attempts with exponential backoff ($800 \cdot 2^n$ ms + jitter) on retryable statuses (408, 409, 429, 500, 502, 503, 529).
- Computes cost from a per-model rate card (input/output/cache-write/cache-read per million tokens) in integer micro-USD, extracts confidence from `data.confidence / overallConfidence`, and writes an `AiCallLog` row on both success and terminal failure.

A streaming variant, `callLLMStream()`, powers the assistant — an async generator yielding text deltas with a best-effort ledger row on completion.

3.2 The fifteen agents

Twelve core agents plus three Surface-C agents. Tiers: **fast** = `claude-haiku-4-5-20251001` , **balanced** = `claude-sonnet-4-6` , **reasoning** = `claude-opus-4-8` .

AGENT	RESPONSIBILITY	TIER	SURFACES
IntakeAgent	Parse free text/PDF into a structured intent (ARR) or document model	balanced	A
RuleMappingAgent	Interpret the ruleset → answerable/cautioned/out-of-scope fields, format, checklist	reasoning	A
RetrievalPlanner-Agent	Turn the plan into a typed query plan (no DB access)	balanced	A
ExtractionAgent	Read documents natively → typed fields + per-field confidence; flag gaps	reasoning	B1, B2
GenerationAgent	Generate the compliant report / onboarding-decision narrative	reasoning	A, B1
ValidationAgent	Adversarial self-check vs checklist; blocks completion on discrepancies	reasoning	A, B1, B2, C
RiskAgent	Explainable partner risk score with contributing factors	balanced	B1, B2
ExpiryForecastAgent	Forecast credential expiries into proactive gap predictions	fast	B2
DriftDetectionAgent	Compare prior vs fresh state → material compliance drift	balanced	B2
ComplianceConstraintAgent	Reason about residency/PII/lawful-basis; explain constraint decisions (deterministic rules override it)	reasoning	middleware
ExplainAgent	Human-readable rationale for a field/decision	balanced	A, B1
AssistantAgent	Grounded conversational Q&A with citations	balanced	global
RegulationReader-Agent	Read a regulation → quotable sections + detected jurisdiction/language	reasoning	C
RegulationClassifierAgent	Classify jurisdiction/regime; new-market vs version-update	balanced	C
RulesetSynthesis-Agent	Synthesize a complete executable ruleset + per-rule provenance + honest gaps	reasoning	C

3.3 Model routing & tiers

Routing is centralized in `engine/llm/router.ts`. The rationale: **reasoning** (Opus) for legal interpretation, document extraction (errors poison everything downstream), generation (format/constraint adherence), and self-validation (must be at least as capable as the generator, run with an adversarial prompt to avoid shared blind spots); **balanced** (Sonnet) for structured mapping, scoring, drift, rationale, and the assistant; **fast** (Haiku) for date-arithmetic-heavy forecasting and simple classification. An `escalate` hint bumps a call up one tier. The rate card (per million tokens) is: Opus \$5 in / \$25 out, Sonnet \$3 / \$15, Haiku \$1 / \$5, each with cache-write and cache-read discounts; tier timeouts are 45s / 90s / 150s.

3.4 The prompt registry & the 17 prompts

Prompts are versioned markdown files under `engine/llm/prompts/` named `<id>.v<n>.md`, each with front-matter (id, version, tier, output schema) and `===SYSTEM===` / `===USER===` sections. The loader caches all versions and defaults to the highest; the version string flows into every result, ledger row and figure, so any artifact is traceable to the exact prompt that produced it. New prompt versions are meant to be regression-gated by the \$7.B eval harness.

PROMPT	TIER	NOTABLE RULES
<code>intake.request</code>	balanced	Extract intent; flag ambiguities <i>only</i> when the reporting period is undeterminable; never flag residency/PII/lawful-basis questions.
<code>rules.interpret</code>	reasoning	Map each requested field to a ruleset key; classify answerable/cautioned/out-of-scope respecting residency and restricted-disclosure keys.
<code>retrieval.plan</code>	balanced	One query per answerable/cautioned field; only provided field keys; never queries the DB.
<code>extraction.fields</code>	reasoning	Extract expected fields with calibrated per-field confidence; null + present=false when illegible; list gaps.
<code>generation.report</code>	reasoning	Use only the computed dataset; copy values exactly; set sourceRowIds to exactly the dataset's ids; mandated language/structure.
<code>generation.onboarding</code>	reasoning	Draft approve / approve-with-conditions / reject; low-confidence extraction → human-verification condition.
<code>validation.selfcheck</code>	reasoning	Adversarial: per-checklist satisfied? per-figure plausible vs sources? blocking on any unresolved discrepancy.
<code>risk.score</code>	balanced	0–100 score from the market risk model factors+weights; per-factor contributions summing to the score; map to band.
<code>risk.forecast</code>	fast	Severity by urgency (≤7d CRITICAL, ≤21d HIGH, ≤30d MEDIUM); concrete impact line.
<code>risk.drift</code>	balanced	Compare snapshots; new vehicles without docs, lapsed docs, fleet change, now-failing under active ruleset.
<code>compliance.constraint</code>	reasoning	Per field/data-class: allow/mask/aggregate/omit/block with governing rule + rationale; deterministic bright-line rules override it.
<code>explain.rationale</code>	balanced	Short plain-language rationale for one field/decision/finding.
<code>assistant.grounded</code>	balanced	Answer only from provided context; cite records; grounded=false if insufficient; always 2–4 suggestions.
<code>assistant.stream</code>	balanced	Markdown answer with inline record links + a trailing <code>===DATA===</code> JSON block (chart/suggestions/grounded).
<code>regulation.read</code>	reasoning	Decompose into stable quotable sections (the provenance source); detect language/country/regulator/topics.
<code>regulation.classify</code>	balanced	Jurisdiction; region; privacy regime; suggested market code; new market vs version update.
<code>ruleset.synthesize</code>	reasoning	

		Emit a full ruleset using only closed vocabularies; unmappable items → unmappedFields/unsupportedClauses; per-element provenance with verbatim quote.
<code>ruleset.validate</code>	reasoning	Adversarial review of the draft ruleset; faithfulness/completeness/coherence/privacy; blocking on critical/high.

3.5 Cost ledger

Every Claude call logs an `AiCallLog` row: agent, stage, model, prompt id and version, tokens in/out, cache-read/write tokens, latency, cost in micro-USD, confidence, a request hash, and an ok/error flag, linked to the authority request / partner / document / pipeline run that triggered it. The Audit & Cost and Home views aggregate cost per request/partner/market/agent and surface it alongside cumulative time-saved as the capital-efficiency story. Cost uses the per-model rate card including the cache-read discount.

PART IV

Shared Core Architecture

The DRY engine that powers all four surfaces: module map, the re-entrant pipeline runner, the governance state machines, provenance, the hash-chained audit log, and the storage driver.

4.1 Engine module map

Bolt Sentinel is a shared compliance-intelligence core with thin surfaces on top. The surfaces are pipeline configurations of one runner.

MODULE	RESPONSIBILITY
<code>engine/llm/</code>	Centralized Anthropic client (the only SDK importer), model router, versioned prompt registry, cost ledger.
<code>engine/types/</code>	The single source of truth — Zod schemas + TS unions for all enums, the ruleset contract, and every AI artifact.
<code>engine/rules/</code>	DB-backed runtime ruleset resolver (<code>store.ts</code>), the seed-time registry, the structural diff, the executor dry-run validator, and the six market ruleset files.
<code>engine/retrieval/</code>	The deterministic executor — the only Prisma caller in the AI path; computes figures with real provenance.
<code>engine/validation/</code>	The declarative rule-DSL evaluator (validations + cross-document checks).
<code>engine/compliance/</code>	§7.A constraint framework: residency, PII, lawful-basis, retention, purpose limitation.
<code>engine/accuracy/</code>	§7.B ground-truth eval harness + figure-vs-source verification.
<code>engine/governance/</code>	Hash-chained audit log + the four human-in-the-loop state machines.
<code>engine/pipeline/</code>	Stage/Context types, the re-entrant runner, the pipeline registry, progress emission.
<code>engine/storage/</code>	Document driver — local filesystem in dev, Vercel Blob in prod.

4.2 The re-entrant pipeline pattern & runner

A surface defines `Pipeline = { surface, stages[] }`, where each `Stage` is `{ name, label, run(ctx, io) }`. Contexts are **re-entrant**: they carry only entity ids and metadata, and each stage re-reads prior results from the DB and persists its own — so the chain is independently resumable, one stage per invocation.

The runner (`engine/pipeline/runner.ts`):

- **startPipeline** creates a `PipelineRun` (QUEUED) wired to the entity by surface, creates ordered `PipelineStage` rows (PENDING), moves the entity to PROCESSING (B2 excepted), and audits `PIPELINE_START`.
- **advanceOneStage** runs exactly one pending stage. It *atomically claims* the stage with an `updateMany(PENDING→RUNNING)` — if the claim count is zero (a concurrent advance already claimed it) it returns without re-running, preventing double execution and double AI spend. On a `blocked` result the stage becomes BLOCKED, the run becomes AWAITING_REVIEW, the entity is routed to PENDING_REVIEW, and a `PIPELINE_GATE_OPENED` audit (actor AI) is written. On success the stage is DONE; on throw it is FAILED and the run FAILED.
- **finishRun** completes the run (recording `elapsedMs` and the entity's `aiElapsedMs` for the time-saved contrast) and transitions the entity to PENDING_REVIEW.

WHY ONE STAGE PER INVOCATION

Vercel functions have a bounded duration, so no single invocation runs a whole multi-stage AI pipeline. Completed stages are immutable and persisted, so a timeout merely pauses the chain; a resume sweeper (`/api/cron/resume-pipelines`) finds stalled runs, resets stuck RUNNING stages to PENDING, and continues from the first non-done stage. This also gives the live per-stage progress the UI streams.

4.3 Governance state machines

Four typed transition maps (`engine/governance/state.ts`) make an illegal status string impossible to write — `assertTransition` throws on any illegal move, and every transition writes an audit row. No binding, submitted, or suspended state is ever reached without a human action.

MACHINE	TRANSITIONS (ABRIDGED)
ARR	RECEIVED → PROCESSING → PENDING_REVIEW → (APPROVED REJECTED NEEDS_CLARIFICATION) → DONE
Partner (B1)	RECEIVED → PROCESSING → PENDING_REVIEW → (APPROVED CONDITIONS_APPLIED REJECTED); terminal states reachable only via a human.
Monitoring (B2)	COMPLIANT → (EXPIRING_SOON DRIFT_DETECTED) → PENDING_REVIEW → (REMEDIATED CONDITIONS_APPLIED SUSPENDED_RECOMMENDED) → COMPLIANT
RegImport (C)	RECEIVED → PROCESSING → PENDING_REVIEW → (ACTIVATED REJECTED NEEDS_EDIT) → DONE

4.4 Provenance system

The deterministic executor returns every record tagged with its primary key and the query that fetched it. The GenerationAgent cites only those row ids (Structured Outputs + a deterministic post-check). Each figure is persisted as a normalized `ReportField` carrying `{ fieldKey, label, value, classification, confidence, verified, provenance[], aiCallId }`. The §7.B layer recomputes the figure from the cited rows to set `verified`. The UI renders each figure as a click-through to its source rows plus the exact AI call (model, prompt version, cost, confidence) that produced it.

4.5 Hash-chained audit log

Every meaningful action — AI call, constraint decision, gate open/close, human decision, state change — is appended to `AuditLog` with actor, action, entity, before/after and timestamp, and for AI rows the model, prompt version, cost and confidence. Rows are hash-chained: `hash = sha256(prevHash + canonical(payload))` using a deterministic sorted-key JSON, so any tampering breaks the chain downstream. `verifyAuditChain()` recomputes every hash in order and reports the first break; the Audit view shows a "Verified" / "Broken @ N" integrity badge.

4.6 Document storage driver

A storage-driver interface (`put` / `getBytes` / `getUrl` / `delete`) abstracts over a local-filesystem implementation (dev) and a Vercel Blob implementation (prod), selected by environment. `Document.storageRef` holds the key (local) or the full public URL (blob). PDFs are generated at seed time (Vercel's filesystem is ephemeral/read-only), and bytes are fetched as base64 to feed Claude's native PDF/image input. A guarded route (`/api/documents/[...path]`) streams stored bytes with the correct content type.

PART V

The Two Critical Subsystems

Governance and correctness are architectural, not bolted on: the §7.A Compliance-Constraint Framework and the §7.B Accuracy & Confidence Layer.

5.1 §7.A — Compliance-Constraint Framework

"How is governance enforced?" has a structural answer. Each market ruleset carries a typed `compliancePolicy` ; middleware enforces it, and every constraint decision is explainable and audited.

The framework (`engine/compliance/checks.ts`) enforces bright-line rules deterministically; the `ComplianceConstraintAgent` reasons about and explains decisions but **cannot override a hard block**. The authoritative `deterministicAction` resolves, per field:

FIELD CLASSIFICATION	ACTION	RULE
OUT_OF_SCOPE	block (withheld)	Restricted / special-category disclosure — overreach. Field omitted from the dataset.
CAUTIONED + list value + (restricted or residency-restricted)	aggregate	PII minimization / cross-border residency — the list value is replaced by its count, relabelled "(aggregated count)."
CAUTIONED (otherwise)	allow	Disclosure permitted with caution.
ANSWERABLE	allow	Legal obligation.

Residency-restricted means the policy's `crossBorderTransferAllowed` is false; restricted means the field is in the policy's `restrictedDisclosureFieldKeys` . The framework also supports masking (stripping name/licenseNo per row). The LLM enrichment adds a human-readable rationale per field but the deterministic action stands; failures of the LLM call are swallowed so the deterministic decision always holds. The policy covers data residency (storage region, cross-border allowance, allowed transfer regions), PII classification per field, lawful bases (tag → applicable field keys), purpose limitation (declared purposes + restricted-disclosure keys), and retention (document/audit months + auto-purge). This drives the Lagos NDPA residency scenario and the Johannesburg POPIA overreach scenario.

5.2 §7.B — Accuracy & Confidence Layer

"How do you know the AI is right?" has a visible answer, because synthetic documents are generated *from* known ground truth — so accuracy is measurable.

Per-field confidence & gating

Every extracted field and generated figure carries a confidence score. Gating thresholds live once in `lib/confidence.ts` : pass ≥ 0.85 (higher for sensitive 0.92 / financial 0.90 fields), review 0.60–0.85, and block below 0.60. Critically, `gateDecision` **always blocks** an unverified figure or a validation mismatch regardless of confidence. The gate is wired into the human-in-the-loop checkpoints.

Figure-vs-source verification

`engine/accuracy/verify.ts` recomputes each report figure from its cited rows, independent of the LLM — the hard backstop against hallucinated numbers. A figure must (a) match a dataset field, (b) cite a non-empty set of source row ids that is a subset of the dataset's provenance ids, and (c) have a matching value. Any failure marks the figure unverified and should block completion.

Ground-truth eval harness

`engine/accuracy/eval.ts` measures extraction precision/recall/F1 against the documents' ground-truth answer keys, builds a calibration reliability view (predicted confidence vs actual accuracy per bucket), tracks per-field confidence distribution, computes per-market F1, and measures the defect catch-rate — whether injected low-legibility defects are caught by low confidence. Confidence buckets are 0–50% / 50–70% / 70–85% / 85–100%. All of this is surfaced in the Accuracy view, making correctness visible — a credibility multiplier for a governance-grade product.

PART VI

The Market Ruleset System

The "market = ruleset" abstraction, the MarketRuleset contract, the validation DSL and executor capability surface, and a detailed tour of all six markets.

6.1 Market = ruleset abstraction

The engine never references a market by name. Every rule lives in the DB as a Zod-validated `MarketRuleset` in the `RegulatoryRuleset.content` column. The runtime resolver `engine/rules/store.ts` reads rules from the DB (cached by immutable `code:version`, with the mutable active-version pointer always read fresh). The built-in six markets are *seeded into* the DB from versioned TypeScript files under `engine/rules/markets/<CODE>/vN.ts` — code is seed input only. Adding or changing a market is a new ruleset version, produced either by seed or by the Regulation Intake surface, never by changing engine code.

6.2 The MarketRuleset contract

A ruleset (`engine/types/ruleset.ts`) is a single Zod-validated object with these blocks:

BLOCK	CONTENTS
Identity & locale	<code>marketCode</code> , <code>version</code> , country, regulator (name/code/website), region, timezone, currency, locale, zones[].
<code>requiredDocuments[]</code>	Per doc: type, label, which partner types require it, optional flag, expected fields (key/label/type/PII class), validation rules, and validity months.
<code>crossDocumentChecks[]</code>	Compare a field on document A vs a field on document B (CROSS_FIELD_EQUALS or MATCHES_REGEX) with severity and a fail message.
<code>authorityFields[]</code>	The authority-answerable catalogue: key, label, classification (answerable/cautioned/out-of-scope), data class, an executor <code>source</code> (entity + aggregation + field + filter rule ids), caution note, lawful-basis tag.
<code>reportFormat</code>	Mandated format: id, mandated-by, delivery (PDF/JSON/both), sections (heading + field keys), header/footer, date format, number format, language.
<code>compliancePolicy</code>	§7.A: privacy regime, data residency, PII classification, lawful bases, purpose limitation, retention.
<code>namedFilters</code>	Reusable filter fragments (e.g. <code>active</code> , <code>license_valid</code> , <code>inspection_valid</code>).
<code>riskModel</code>	Weighted factors and the band thresholds (low/medium/high).

6.3 Validation DSL & executor capability surface

Validations use a declarative DSL of 13 operators — EXISTS, NOT_EMPTY, EQUALS, NOT_EQUALS, MATCHES_REGEX, DATE_NOT_EXPIRED, DATE_WITHIN_DAYS, GTE, LTE, GT, LT, IN_SET, CROSS_FIELD_EQUALS — evaluated deterministically (`engine/validation/rules.ts`) with flexible date parsing (ISO and DD.MM.YYYY / DD/MM/YYYY) and case-insensitive comparison. So even complex per-market validations need no engine code.

Authority-field sources must be executable by the deterministic executor (`engine/retrieval/executor.ts`); a static validator (`validate-sources.ts`) keeps the ruleset in lockstep with it. The executor's capability surface:

ENTITY	AGGREGATIONS	FILTERABLE / SELECTABLE FIELDS
Driver	count, list	licenseExpiresAt, permitExpiresAt, status, nationalId, licenseNo, permitNo, fullName (named filters supported)
Vehicle	count, list	registration/insurance/inspection validity dates, status, addedAt, plate, year, make, model, vin, registrationNo
Trip	count, sum	zone, city, startedAt, fareMinor, distanceKm (scoped by request zone/period, not named filters)
FleetPartner	count, list	partnerType, status, riskBand, riskScore, reference, legalName, tradingName, registrationNo

The executor scopes every query to the market, applies named filters and request period/zone, caps provenance id arrays at 100, and reads a synthetic "as of" date from app metadata so seeded expiry/anomaly scenarios are stable.

6.4 The six markets in detail

All six share a common risk model (completeness 0.30, expiry proximity 0.25, inconsistencies 0.30, fleet size 0.15; bands low 0–33 / medium 34–66 / high 67–100) and a standard answerable-field set (active drivers, drivers with valid license, per-driver list, completed trips scoped by zone/period, active vehicles, vehicles with valid inspection, fleet-partner list). The differences are the regime, documents, formats and constraints below.

Estonia — EE_TALLINN (GDPR, region EU)

Regulator Transpordiamet (TRAM). EUR, locale `et`, Europe/Tallinn, 8 Tallinn zones. Documents include an operator permit (`^EE-TVR-\d{6}$` , issuer must be Transpordiamet, 60-month validity), 8-digit business registry code, vehicle insurance/registration/inspection, and driver license. Cross-checks: permit holder vs company name (HIGH), insurance holder vs company (MEDIUM). Report `EE_TRAM_STANDARD_V1` , both delivery, date DD.MM.YYYY. Cross-border allowed within the EU; no restricted disclosure keys.

Version 2 adds a required `DRIVER_BACKGROUND_CHECK` (criminal-record extract, a SENSITIVE field, result must be non-empty) for COMPANY / FLEET_OPERATOR partners — this is the B2 ruleset-update trigger that re-flags v1-compliant partners.

Poland — PL_WARSAW (GDPR, region EU)

Regulator Prezydent m.st. Warszawy (UMSTW). PLN, locale `pl` . Distinctive: a taxi license (`^WA/TAXI/\d{4}/\d{4}$`), a license extract with a Warsaw plate, a 10-digit NIP business registration, a vehicle registration requiring a taxi annotation, and a 1-month-validity driver criminal record. Cross-checks: licence holder vs company (HIGH), extract references the parent licence (MEDIUM). Report `PL_UMSTW_STANDARD_V1` .

Portugal — PT_LISBON (GDPR, region EU)

Regulator IMT. EUR, locale `pt-PT` . TVDE regime: operator license (`^TVDE-\d{6}$` , 120 months), TVDE driver certificate (CMTVDE, 60 months), vehicle TVDE badge linked to the operator licence number, vehicle inspection (IPO), insurance, and a 9-digit NIF business registration. Cross-checks: operator vs company (HIGH), badge references operator licence (MEDIUM). Report `PT_IMT_STANDARD_V1` , date DD/MM/YYYY.

Romania — RO_BUCHAREST (GDPR, region EU)

Regulator ARR. RON, locale `ro` , 6 sectors. Distinctive: a 6-month vehicle ITP inspection cycle, an alternative-transport authorization (`^ARR-B-\d{6}$`), a driver attestation (atestat), RCA insurance, vehicle registration, and a CUI business registration (`^R0\d{2,10}$`). Cross-checks: authorization holder vs company (HIGH), insurance plate vs registration plate (MEDIUM). Report `RO_ARR_STANDARD_V1` .

Nigeria — NG_LAGOS (NDPA, region NG — non-EU)

Regulator Lagos State MoT / LASDRI. NGN, locale `en` , Africa/Lagos. Documents: a service-entity permit (`^LSG-SEP-\d{6}$`), operator provisional license, vehicle roadworthiness, insurance, a 24-month LASDRI certificate, and an RC business registration. **Distinctive compliance:** NDPA regime, storage region NG, *cross-border transfer not allowed* (NDPA s.43), and two CAUTIONED fields (driver personal-details list and fleet-partners list, both in restricted-disclosure keys with NDPA minimization caution notes). Retention 24/60 months with auto-purge enabled. Report `NG_LAGOS_STANDARD_V1` . This drives the Lagos NDPA aggregation/masking scenario.

South Africa — ZA_JOHANNESBURG (POPIA, region ZA — non-EU)

Regulator NPTR / Gauteng PRE. ZAR, locale `en`, Africa/Johannesburg. Documents: an operating licence under the NLTA (`^NPTR-[A-Z]{2}-\d{6}$` , 84 months), NPTR registration certificate, a professional driving permit (PrDP), vehicle roadworthiness (CoR), insurance, and a CIPC business registration (`^\d{4}/\d{6}/07$`). **Distinctive compliance:** POPIA regime, storage region ZA, cross-border not allowed (s.72); the driver personal-details list is CAUTIONED, and the *driver criminal-record list* is `OUT_OF_SCOPE` (special-category, POPIA s.26, lawful basis consent) — the explicit overreach test case. Retention 36/60 months, auto-purge. Report `ZA_NPTR_STANDARD_V1` . This drives the Johannesburg overreach scenario.

PART VII

Data Model

The full Prisma schema — SQLite locally, Turso (libSQL) in production via the driver adapter (same `sqlite` provider).

7.1 Conventions

UUID primary keys, `createdAt` / `updatedAt` , soft-delete (`deletedAt`), statuses/types as `String` backed by Zod unions in `engine/types/` , arrays as JSON, money as `Int` micro-USD. Because Turso is libSQL there are no native enums and no scalar arrays — anything filtered or aggregated is a real column; JSON is only for display/provenance payloads.

7.2 Models

MODEL	PURPOSE & KEY FIELDS
Market	A market: code, country, ISO2, cities, regulator name/code, privacy regime, region, timezone, currency, locale, and <code>activeRulesetVersion</code> (the mutable pointer).
RegulatoryRuleset	A versioned ruleset: version, filePath, contentHash, summary, changelog, the full Zod-validated <code>content</code> (the DB source of truth), source (SEED/IMPORT), status (ACTIVE/SUPERSEDED), isActive. Unique on (marketId, version).
RulesetImport	Surface C: the uploaded regulation + AI artifacts (reading, classification, draftRuleset, validation, diff), target market/version, status, and activation results. The draft is quarantined here until activation.
AuthorityRequest	Surface A: reference, market, ruleset version, title, authority, legal basis, source text/PDF, the AI artifacts (intent, compliancePlan, retrievalPlan, retrievalResult, generatedReport, selfValidation), status, timing (received/deadline/processing/decided, manual baseline, aiElapsedMs), reviewer, and related reportFields/aiCalls/validations/pipelineRuns.
ReportField	A normalized generated figure: fieldKey, label, value, classification, confidence, <code>verified</code> , <code>provenance[]</code> , <code>aiCallId</code> , ordering. Powers the click-through provenance UI.
FleetPartner	B1/B2: reference, market, legal/trading name, type, registration, contacts, address; onboarding (status, decision, riskScore, riskBand); monitoring (monitoringStatus, reason, lastAssessedAt/ RulesetVersion, completenessPct); timing/baseline; related documents/drivers/vehicles/trips/risk/ validations/crossChecks/events/history/aiCalls/pipelineRuns.
PartnerHistory	Lifecycle events: ONBOARDED, REVALIDATED, DRIFT, REMEDIATION, CONDITION, SUSPENSION, RULESET_REFLAG with detail and timestamp.
Document	A partner/market document: type, title, storageRef, mime/size/sha256, extractedFields, validation-Result, extractionConfidence, status (UPLOADED → ... → VALIDATED/FLAGGED/REJECTED), issued/expires, and its ground truth.
DocumentGroundTruth	The known answer key behind a synthetic document: fields, injectedDefects, generatorSeed — the basis for the \$7.B eval harness.
Driver / Vehicle / Trip	Operational entities under a partner: drivers (license/permit numbers + expiry, status), vehicles (plate, VIN, make/model/year, registration/insurance/inspection validity, status, addedAt), trips (driver/vehicle, started/ended, distance, fare in minor units, zone, city).
Validation / CrossCheck	B1/B2 results: a validation (ruleId, scope, outcome PASS/FAIL/WARN/NOT_APPLICABLE, severity, message, evidence, confidence) and a cross-check (checkId, two docs, field, valueA/valueB, outcome MATCH/MISMATCH/INDETERMINATE, confidence).
RiskAssessment	A risk snapshot: ruleset version, score, band, factors, explanation, <code>isCurrent</code> .
ComplianceEvent	Monitoring events: type (STATE_CHANGE, EXPIRY_FORECAST, DRIFT_DETECTED, RENEWAL, REVALIDATION, RULESET_REFLAG, HUMAN_ACTION, ALERT), severity, title, detail, occurred/due/resolved.
PipelineRun / PipelineStage	The live stepper's source: a run (surface, linked entity, status QUEUED → RUNNING → AWAITING_REVIEW → COMPLETED/FAILED, currentStage, elapsedMs) and its ordered stages (status, progressPct, note, output, confidence, blockReason, durationMs). Unique on (runId, ordering).

AuditLog	Append-only, integer-ordered, hash-chained: actor (HUMAN/AI/SYSTEM), action, entity, before/after, AI metadata (model, promptVersion, costMicroUsd, confidence, aiCallId), prevHash, hash, timestamp.
AiCallLog	The cost ledger: linked entity/run, agent, stage, model, promptId/Version, tokens in/out, cache tokens, latency, costMicroUsd, confidence, requestHash, ok/error.
User / Session	Auth: user (email, name, bcrypt passwordHash, legacy role, assigned <code>roles[]</code> , isActive) and session (hashed token, expiry).
AppMeta	Key/value JSON store for settings, the secret bag (Anthropic key, API token hashes), and the synthetic <code>seedAnchor</code> date.

PART VIII

API Reference

Every route handler. All run on the Node runtime, force-dynamic. Authenticated routes use `guard()` (returns the session user or a 401); a handful add explicit role/permission checks.

8.1 Response envelope

Handlers return `{ ok: true, data }` or `{ ok: false, error }` with an appropriate status. Human-in-the-loop is structurally enforced: no API path lets AI reach a binding state — APPROVED/REJECTED/CONDITIONS_APPLIED/SUSPENDED_RECOMMENDED/ACTIVATED occur only in the decision/activate/reject handlers via a HUMAN actor.

8.2 Endpoints

METH- OD	PATH	PURPOSE
POST	<code>/api/auth/login</code>	Email/password login; sets session + role cookies.
POST	<code>/api/auth/logout</code>	Deletes the session, clears cookies.
POST	<code>/api/auth/role</code>	Switch the active role (must be assigned); audits the switch.
POST	<code>/api/assistant</code>	Grounded structured answer.
GET	<code>/api/assistant/preview?href=</code>	Live record popover for a citation.
POST	<code>/api/assistant/stream</code>	Token-by-token streamed answer + trailing data block.
POST	<code>/api/authority-requests/[id]/process</code>	Start (or resume) the ARR pipeline (202 + runId).
POST	<code>/api/authority-requests/[id]/decision</code>	Human action: approve / reject / request_changes / regenerate / clarify.
GET	<code>/api/authority-requests/[id]/export?format=</code>	Export the report as PDF or JSON.
POST	<code>/api/partners/[id]/process</code>	Start (or resume) the B1 onboarding pipeline.
POST	<code>/api/partners/[id]/decision</code>	Human onboarding decision: approve / approve_with_conditions / reject / regenerate.
POST	<code>/api/partners/[id]/monitoring/decision</code>	Human monitoring action: remediate / apply_conditions / recommend_suspend / clear.
POST	<code>/api/partners/[id]/monitoring/revalidate</code>	Re-extract renewed documents and re-run monitoring.
POST	<code>/api/monitoring/sweep</code>	Run the monitoring sweep across the portfolio (or one market).
GET	<code>/api/cron/monitoring-sweep</code>	Cron-guarded portfolio sweep.
GET	<code>/api/cron/resume-pipelines</code>	Cron-guarded resume of stalled runs.
POST	<code>/api/pipelines/[runId]/advance</code>	Advance one pipeline stage.
GET	<code>/api/pipelines/[runId]/status</code>	Run + per-stage status for the live stepper.
POST	<code>/api/regulation-intake/process</code>	Upload a regulation (file or text); start Surface C.
POST	<code>/api/regulation-intake/[id]/activate</code>	The only path a synthesized ruleset enters the rules table; re-flags affected partners.
PATCH	<code>/api/regulation-intake/[id]/draft</code>	Save reviewer edits to the quarantined draft (schema-gated).
POST	<code>/api/regulation-intake/[id]/reject</code>	Reject the import.
GET		Resolve a figure to its source rows + originating AI call.

	<code>/api/report-fields/[id]/proven- ance</code>	
GET	<code>/api/documents/[...path]</code>	Stream a stored document by storage ref.
PATCH	<code>/api/settings</code>	Save a settings section (schema-validated).
POST	<code>/api/settings/anthropic-key</code>	Set/clear the Anthropic key (ADMIN only; never echoed).
POST / DE- LETE	<code>/api/settings/api-tokens</code>	Create (reveal-once) / revoke an ingest token (ADMIN only).
POST	<code>/api/settings/audit-verify</code>	Verify the hash-chained audit log.
POST / PATCH	<code>/api/settings/users</code>	Create / update users & roles (requires <code>users.manage</code>).

PART IX

Frontend, Design System & UX

The Bolt-styled design language, the app shell, navigation, and the shared component library that make Sentinel look like an internal operations console.

9.1 Design tokens

Design tokens live in `app/globals.css` (Tailwind v4). The brand green is `#34D186` (full 50–950 scale + hover/active), with light green `#BDF4D3`, near-black ink `#1A1A1A`, layered surfaces (white / `#f8f9fa` / `#f1f3f4`), and status colours (success green, warning amber, danger red, info blue) each with muted/foreground variants. Typography is Inter (with Geist Mono for data), 4px button / 8–12px card radii, a 4px spacing base, and a subtle two-tier shadow scale. The product ships **dark by default** (the theme is read server-side from a cookie so there is no flash), with a full light theme and a login brand-rail token set. These values were researched from Bolt's live surfaces (Wikipedia brand colours, bolt.eu using Radix + Inter) and recorded in `docs/DECISIONS.md`; the lightning-bolt mark is an original SVG recreation per the IP guardrail.

9.2 App shell & navigation

The console shell (`components/app-shell/`) is a sticky left sidebar plus a top bar. The sidebar shows the logo lockup, permission-filtered nav grouped into **Operations** (Home, Authority Requests, Fleet Onboarding, Compliance Monitoring) and **Governance** (Markets & Rules, Regulation Intake, Accuracy, Audit & Cost, Assistant), pinned utility links (Settings, User Guide), and an account dropdown. The top bar shows the active section, a **market filter** select that writes `?market=` into the URL, an "Ask" button (⌘J) for the assistant, a theme toggle, and a role switcher (a dropdown when the user has multiple assigned roles, switching the active role via the API). The command palette (⌘K) and the floating assistant dock are mounted globally.

9.3 Shared component library

Built on shadcn/ui primitives (accordion, avatar, badge, button, card, checkbox, collapsible, command, dialog, dropdown-menu, input/input-group, label, popover, progress, scroll-area, select, separator, sheet, skeleton, sonner, switch, table, tabs, textarea, tooltip). Sentinel-specific shared components include:

- **PipelineStepper** — starts a pipeline, polls `/status` every 700ms, drives `/advance` one stage at a time (up to 14), shows expandable per-stage output and confidence, and refreshes on completion.
- **StatusBadge** — a large status → tone/icon/label map with a spinner for processing states; **SlaCountdownChip** — a live countdown (neutral → amber ≤24h → red overdue); **ConfidenceMeter** — % + band label + bar (never colour-only); **DecisionOutcome** and **TimeSavedBanner/Chip**.
- **Assistant suite** — AssistantDock (floating, animated), ChatThread (streaming bubbles, starter/follow-up chips, markdown + chart + inline citations), the streaming chat hook, CommandPalette, ChatChart (Recharts bar/line/donut), MessageMarkdown (react-markdown + remark-gfm with record-link rewriting), and RecordLink (popover preview).
- **Panels, KpiStat, PageHeader, EmptyState, skeleton loaders**, the SweepButton, the theme toggle, and the original Bolt-style Logo/BoltMark.

Polish details: skeleton loaders during AI processing, live per-stage progress, smooth transitions, empty/error/not-found states, and a reduced-motion media query. Server data fetching is via Next.js server components; client state uses TanStack Query (5s stale time, retry once) with sonner toasts.

PART X

RBAC, Authentication & Security

Seven roles, sixteen permissions, session-based auth, and the tamper-evident audit chain.

10.1 Authentication & sessions

Auth is session-based (`lib/auth.ts`): an opaque 32-byte token in an httpOnly cookie (`sentinel_session`), sha256-hashed at rest with a server secret; the active role is held in `sentinel_role` ; sessions last 30 days; passwords are bcrypt-hashed. The session user carries a legacy derived role, the assigned RBAC roles, the active role, and the permissions derived from that active role.

10.2 Roles & permissions

Seven roles (`lib/rbac.ts`): **admin**, **compliance_reviewer**, **onboarding_officer**, **compliance_monitor**, **regulatory_author**, **authority_liaison**, **auditor**. Sixteen permissions span home, ARR (view/review/export), onboarding (view/decide), monitoring (view/act), rules (view/author), accuracy, audit, assistant, settings (view/manage), and users.manage. Admin holds all; each other role holds a focused subset (e.g. `authority_liaison` gets ARR view/export, `auditor` gets read-only across surfaces). Nav links are gated by a permission map, and the Settings RBAC matrix renders the roles × permission-group grid.

ENFORCEMENT NOTE

Fine-grained RBAC is enforced at the navigation/UI layer and on the settings routes (ADMIN / `users.manage`). Most action routes enforce authentication via `guard()` ; surface-level permission gating lives in the nav layer. The binding-state invariant (no AI-driven approval/suspension/activation) is enforced independently in the handlers and state machines.

10.3 Audit & secrets

Every action is appended to the hash-chained audit log (Part IV); the chain is verifiable on demand. Secrets (the Anthropic API key, ingest token hashes) live in a separate secret bag in `AppMeta` and are never sent to clients — the key endpoint returns only source + last-4, and ingest tokens are revealed once at creation and stored only as sha256 hashes.

PART XI

Synthetic Data, Seed & Scenarios

How the six markets, partners, documents, ground truth, and the demonstration scenarios are generated — and the operator scripts around them.

11.1 Seeded data

The seed (`prisma/seed.ts` + `prisma/seed-lib/`) creates the six markets from their registry rulesets (Estonia starting on v1, with v2 created inactive), two demo users (`admin@bolt.eu` and `reviewer@bolt.eu` , password `sentinel`) with assigned RBAC roles, and a base portfolio of five approved/compliant partners per market. Each partner gets localized names, 2–4 drivers, 2–5 vehicles with market-correct plate formats and validity windows, 30–80 trips over the last 180 days biased to central zones, and the market's required documents rendered as realistic synthetic PDFs.

Ground truth & synthetic PDFs

Every document is rendered from a known ground-truth field map (`seed-lib/data.ts`) into a full A4 certificate (`seed-lib/pdf.ts`): seven document kinds with per-kind styling, emblems, validity badges, particulars grids, conditions, and an official-looking footer marked "Synthetic document — not a genuine certificate." Defects can be injected — EXPIRED, NAME_MISMATCH, and LOW_LEGIBILITY (which faintly garbles ~35% of characters to genuinely lower extraction confidence) — and stored alongside as `DocumentGroundTruth` , the basis for the \$7.B eval harness.

11.2 The demonstration scenarios

The seed creates the §11 scenarios that make each surface shine — six ARR requests, four B1 onboarding partners, four B2 monitoring situations, and the Estonia ruleset bump — described per-surface in Part II. A synthetic "as-of" anchor date (default 2026-06-04) is stored so expiries and ambiguities land deterministically.

11.3 Operator scripts

SCRIPT	WHAT IT DOES
<code>prisma/warm.ts</code>	Drives all scenarios through the live pipelines so dashboards open populated (costs real tokens).
<code>prepare-full-demo.ts</code>	Resets partners to RECEIVED, clears artifacts, and injects fresh expiry/drift data for a clean demo run.
<code>backfill-b1-artifacts.ts</code>	Recomputes validations/cross-checks/risk/decision for already-decided partners using the real DSL evaluators.
<code>restore-monitoring-portfolio.ts</code>	Restores the B2 portfolio with a realistic risk distribution while keeping a couple of RECEIVED partners per market for B1 demos.
<code>rerender-docs.ts</code>	Re-renders every document PDF with the type-specific renderer, preserving DB state.
<code>validate-rulesets.ts</code> / <code>verify-*.ts</code>	Load every ruleset through the registry, and end-to-end verifiers that drive ARR / B1 / regulation-intake / assistant live and print stage results, figures, confidence and per-call cost.

PART XII

Tech Stack, Deployment & Operations

The production stack, the Next.js 16 conventions the codebase follows, and how it deploys and runs.

12.1 Stack

Next.js 16 (App Router) · TypeScript (strict) · Tailwind v4 · shadcn/ui · Recharts · TanStack Query · Prisma 6 · SQLite (dev) / Turso libSQL (prod) via the Prisma driver adapter · Anthropic Claude (Haiku/Sonnet/Opus tiers, native PDF/image, Structured Outputs) · Vercel Blob storage · pdfkit · bcryptjs · Zod 4 · React 19. Deployable on Vercel.

12.2 Next.js 16 conventions

The codebase follows the bundled Next.js 16 rules (`AGENTS.md`): Turbopack is the default (no custom webpack config), all request-time APIs are async (`await` on `cookies()` , `headers()` , `params` , `searchParams`), DB/session routes are `runtime = "nodejs"` + `dynamic = "force-dynamic"` (Prisma, the Anthropic SDK and pdfkit need the Node runtime, never Edge), long work uses `after()` to schedule the next pipeline stage after the 202 response, and auth middleware uses the `proxy.ts` convention.

12.3 Deployment & ops

Local: `npm install`, set `ANTHROPIC_API_KEY`, `npx prisma migrate dev`, `npm run db:seed`, `npm run dev`.
Production (Vercel + Turso + Blob): set `TURSO_DATABASE_URL`, `TURSO_AUTH_TOKEN`, `BLOB_READ_WRITE_TOKEN`, `SESSION_SECRET`, `ADMIN_TOKEN`, `CRON_SECRET` and `SEED_ANCHOR`, apply the schema to Turso, deploy, and seed via the admin endpoint. Two Vercel crons run the B2 monitoring sweep and the stalled-pipeline resume sweeper; both are also triggerable in-app.

PART XIII

Appendices

Status enumerations, a glossary, and the repository map.

13.1 Status enumerations

DOMAIN	VALUES
ARR status	RECEIVED, PROCESSING, PENDING_REVIEW, APPROVED, REJECTED, NEEDS_CLARIFICATION, DONE
Partner (onboarding)	RECEIVED, PROCESSING, PENDING_REVIEW, APPROVED, CONDITIONS_APPLIED, REJECTED, NEEDS_CLARIFICATION
Monitoring	COMPLIANT, EXPIRING_SOON, DRIFT_DETECTED, PENDING_REVIEW, REMEDIATED, CONDITIONS_APPLIED, SUSPENDED_RECOMMENDED
Document	UPLOADED, EXTRACTING, EXTRACTED, VALIDATED, FLAGGED, REJECTED
Pipeline run / stage	QUEUED, RUNNING, AWAITING_REVIEW, COMPLETED, FAILED / PENDING, RUNNING, DONE, BLOCKED, FAILED, SKIPPED
Regulation import	RECEIVED, PROCESSING, PENDING_REVIEW, ACTIVATED, REJECTED, NEEDS_EDIT, DONE
Validation / cross-check	PASS, FAIL, WARN, NOT_APPLICABLE / MATCH, MISMATCH, INDETERMINATE
Severity / risk band	INFO, LOW, MEDIUM, HIGH, CRITICAL / LOW, MEDIUM, HIGH
Field classification / constraint action	ANSWERABLE, CAUTIONED, OUT_OF_SCOPE / allow, mask, aggregate, omit, block

13.2 Glossary

TERM	MEANING
Surface	One of the four user-facing workflows (A / B1 / B2 / C) configured over the shared engine.
Agent	A typed LLM operation routed to a model tier and backed by a versioned prompt.
Ruleset	The complete, Zod-validated machine-readable description of a market's compliance rules.
Provenance	The source row ids + prompt version behind every generated figure.
Confidence gating	The \$7.B policy that flags or blocks low-confidence/unverified outputs for human review.
Re-flag	Moving partners back to review when their market's ruleset changes to a newer version.
Time-saved contrast	The manual-baseline vs actual-elapsed device surfaced throughout the product.

13.3 Repository map

```
app/           Next.js App Router – (console) screens, (auth) login, (guide), api/ routes
components/    shared UI (ui/ shadcn primitives; brand/; app-shell/; assistant/; shared/; monitoring/)
engine/
  llm/         centralized Claude client, model router, prompt registry, cost ledger
  intake rules retrieval generation validation risk explain  the AI stages
  compliance/  §7.A constraint framework
  accuracy/    §7.B confidence model, gating, ground-truth eval
  governance/  hash-chained audit, human-in-the-loop state machines
  pipeline/    Stage/Context types, re-entrant runner, progress
  storage/     local-fs + vercel-blob document drivers
  rules/markets/<CODE>/  the six market rulesets (versioned) + registry
  types/       shared Zod schemas + TS unions (single source of truth)
surfaces/      authority-request, fleet-onboarding, fleet-monitoring, regulation-intake, assistant
lib/           db factory, auth/session, rbac, confidence thresholds, formatters, report-pdf, settings
prisma/        schema.prisma, seed.ts, seed-lib/ generators, warm.ts
scripts/       demo/backfill/restore/verify utilities
docs/          ARCHITECTURE.md, DECISIONS.md
```

END OF DOCUMENT

Bolt Sentinel — a complete, governed, AI-native Regulatory Operations Platform: four surfaces, fifteen agents, six markets, two critical subsystems, and human-in-the-loop before anything binds. All data synthetic; regulatory details directionally realistic but illustrative.